Reference-Preserving Speculative Decoding on Consumer Systems: Diagnosis, Repair Scope, and Paid Cost

Author information pending confirmation

Abstract

Speculative decoding changes both execution cost and the numerical history of a quantized runtime. When an application must reproduce a fixed ordinary greedy output, the designer needs to determine where to constrain execution and whether the qualified design pays. We study these decisions using natural donor-state crosses, a phase-scoped attention implementation, and paid ordinary and matched target-only baselines. CUDA and Vulkan crosses distinguish local current-path, retained-state and interaction effects. A separate B570 fork study finds unchanged immediate decisions but different trajectories under sustained alternate continuation. On RTX, a query-layout transformation retains batched projection and feed-forward work; target prefill and continuation form an inclusion-minimal passing set within a tested four-switch family. The frozen design qualifies full responses on 24 project-unused inputs. Generation speed is 1.170 times ordinary and 1.169 times matched target-only execution. Resident-request speed is 1.742 times ordinary but 1.035 times matched: the draft-free path change already gives 1.683 times. Seven tasks regress against matched request time, and one round reverses direction. Wider fixed-capacity drafting and a B570 endpoint repair qualify while losing performance. Separate M4 and 358H cases limit added control and draft-only optimization. The study connects implementation-specific diagnosis, finite qualification and paid comparisons to runtime choices, with repair evidence restricted to the tested Qwen3 family and configurations.

Keywords: speculative decoding, numerical consistency, execution state, runtime design, performance evaluation, consumer systems

1. Introduction

Speculative decoding uses a draft to propose tokens and a target to evaluate several positions at once [1]. On consumer systems, this extra work within a request can amortize target execution when memory capacity and request traffic limit batching across requests. The implementation also changes how the target executes: scalar continuation becomes multi-position verification, cache contents follow a different history, and rejected positions require reconciliation. A valid sampling rule therefore leaves two practical questions open: does the quantized runtime reproduce the required output, and does it reduce the cost of producing it?

We study systems in which ordinary greedy decoding under a fixed runtime configuration defines the reference. Such a requirement arises in regression tests and applications that need repeatable outputs. Reference preservation means matching the registered token sequence or endpoint; it does not establish that the answer is functionally correct or that every intermediate floating-point value is identical. The runtime designer must choose where to enforce consistency and then measure the resulting cost. A constraint can repair one input while breaking another, or recover the output at the expense of the intended acceleration.

The visible token prefix does not determine the necessary constraint. Two executions can emit identical tokens while retaining different key/value (KV) values, pending tokens or sampler history. Their next decision can follow the retained state, the current execution recipe, or an interaction between the two. Conversely, different scores can produce the same immediate greedy decision while later outputs diverge. Distinguishing these cases helps decide whether to constrain state formation, current verification, or both.

Performance attribution requires a second distinction. A repair may improve the target path before a draft is added. Comparing repaired speculation only with ordinary decoding measures the combined deployment change. A matched target-only execution, using the repaired path without a draft, measures how much additional benefit speculation supplies. Both paths must be executed and charged under the same output requirement; acceptance rates and target-call counts cannot replace their measured costs.

Our results make these choices concrete. An RTX continuation intervention repairs one task and regresses another, leaving aggregate reference agreement unchanged. A later phase-scoped attention implementation qualifies full responses on 24 inputs unused in the accessible project history. Its resident-request speed is 1.742× ordinary decoding, but the matched draft-free path already achieves 1.683×; adding speculation gives 1.035×. A distinct B570 implementation preserves its registered endpoint while remaining substantially slower than both draft-free baselines. We organize the study around three questions:

1. **RQ1**, diagnosis: does the observed deviation follow retained state, the current recipe, or their interaction?
2. **RQ2**, repair: which phase constraints preserve the reference within a defined implementation family?
3. **RQ3**, cost: after qualification, what benefit remains against ordinary decoding and the matched target path?

The study contributes three forms of implementation evidence. First, natural donor-state crosses with own-path controls distinguish local state, path and interaction effects on CUDA and Vulkan; a separate same-state B570 fork study tests the limits of immediate agreement. Second, an RTX query-layout transformation retains batched projection and feed-forward work while a complete four-switch factorial identifies a sufficient target-phase scope. Third, paid ordinary/matched/speculative comparisons expose when a repair, wider draft or added control is useful and when a qualified design should be rejected.

These contributions concern runtime design and finite qualification. The crosses are offline experiments, and the repair remains specific to the tested Qwen3 family and configurations. Sections 2 and 3 define the implementation context and measurement contracts; Sections 4–6 answer the three questions. Section 7 draws the resulting design choices and states their limits. The supplement retains campaign details, historical results and the complete evidence trail.

2. Runtime Architecture and Research Context

2.1. Execution as a stateful transaction

A speculative iteration advances a committed prefix through drafting, target verification and reconciliation. The draft proposes up to k tokens; the target evaluates the pending token and proposal positions, often in a caller batch containing several tokens. Acceptance, correction and a possible bonus token determine the committed output. Rejected positions must be removed or reconciled before the next iteration, and stopping follows the registered EOG and cap rule. Figure 1 connects these operations to the state and phase controls studied here.

Target and draft retain distinct state (Figure 1A). KV ownership, logical positions, pending logits or tokens, sampler history and loop counters connect successive calls. A token can already be sampled and accepted into the sampler while still awaiting evaluation into the cache. Repeating or omitting either event changes the transaction. The early intervention frontends therefore preserve pending and sampler state as well as KV. The later B570 candidate binds phases within one target context and live sampler; the measured RTX candidate uses no state transplant.

Microbatch capacity UB limits how the backend partitions a caller batch. It is distinct from k, the occupied query count and the frontend's output reservation. A UB2 caller may contain only one continuation token; increasing k can reduce caller count while increasing the number of micrographs; changing output reservation can alter the recipe without changing UB. These distinctions are needed to interpret both diagnostic crosses and performance counters.

The implementations use pinned llama.cpp releases and private frontends [2]. The late RTX path uses Qwen3-8B and Qwen3-0.6B [3], both Q8_0, with non-flash CUDA attention. Its transformation uses existing ggml views and operations, with no new CUDA arithmetic kernel or acceptance rule. B570 uses Vulkan and a distinct phase-binding frontend. Historical Metal and OpenVINO cases retain their own contracts. Because hardware, backend and frontend vary together, these cases compare execution designs rather than hardware alone.

2.2. Reference preservation and determinism

Our requirement fixes an ordinary path and an output endpoint. A matching token sequence may still have different logits or cache values, while a repeat-stable candidate can consistently disagree with the reference. Numerical-consistency work explains how execution shapes and reductions affect this distinction. Batch-invariant inference constrains reduction strategies across batches [4]. LLM-42 combines decode–verify–rollback, padded verification shapes and replacement of decoded KV entries with verifier entries; it permits different consistent strategies for prefill and decode [5]. Stage-Replay examines fixed-prefix precision controls and bidirectional cache transplantation [6]. Vosti specifies and verifies deterministic system behavior [7], and MarginGate uses margin-triggered verification with repair of confirmed mismatches [8].

Our implementation uses these established consistency and verification principles. Its additional evidence links natural donor-state diagnosis to a concrete query-layout and phase design, followed by finite native-output qualification and paid comparisons. Table 1 summarizes this scope. We do not implement a same-contract comparison with LLM-42, Stage-Replay or Vosti, so the results establish no speed or preservation advantage over those methods.

2.3. Performance and heterogeneous execution

Heterogeneous placement and draft-budget control offer established ways to improve execution. Dovetail and DuoDecoding exploit complementary CPU/GPU capabilities [11, 12]; TurboSpec and BanditSpec adapt speculation to execution conditions [13, 14]. The published HeteroSpec studies contextual heterogeneity in verification allocation [15]. The present study addresses a different design requirement: preserving a selected ordinary output while deciding whether the transformation pays.

Lossless but Not Free examines consumer backend cost and cache choreography when verification fails to amortize target work [9]. Performance or Illusion? shows why reductions in apparent decoding work do not guarantee production-engine performance [10]. We add reference qualification and the matched draft-free comparator to that decision. M4 control and 358H offload results supply separate examples of cost limits, rather than replications of the RTX repair.

3. Study Design and Measurement Contracts

3.1. Systems, inputs, and evidence roles

The principal cases use the contracts in Table 2. All final targets are Qwen3-8B: changing the device, draft representation or backend does not create a second target family. HumanEval [16] and sanitized MBPP [17] supply coding inputs; M4 also uses a prospective Spec-Bench partition [18]. Prompt construction remains campaign-specific. The measured output property is reference reproduction, not benchmark test-pass accuracy.

The late RTX and B570 candidates use non-flash attention, greedy sampling and separately frozen capacities, phase settings and runtime/model bindings (Supplement §S1 and §S5.2). Their campaigns are not pooled. We distinguish five evidence

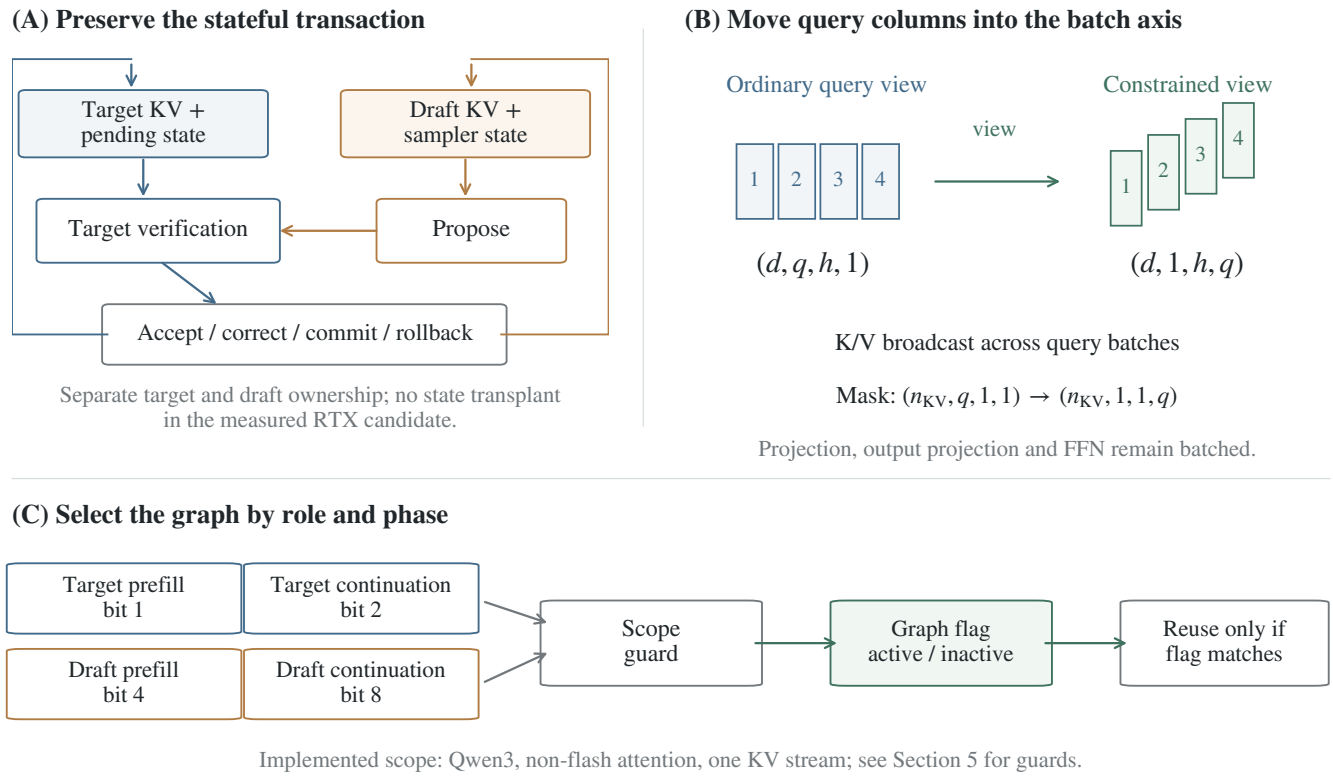


Figure 1: State ownership and implemented consistency controls. (A) Target and draft keep distinct state through proposal, verification and reconciliation. (B) The RTX query view moves query columns into the batch axis while retaining batched projection and FFN work; numbered tiles are illustrative. (C) Role and phase select the query-batch graph flag, which must also match for topology reuse. The measured RTX candidate uses no state transplant. Section 5 defines the transformation and its implementation guards.

roles: prediction reproduces native behavior; selected-boundary crosses diagnose local dependence; exposed probes select a repair; a frozen recipe is checked on project-unused inputs; and separate paid sessions measure qualified cost. Repetitions describe stability and cost variation. Forks and within-request decision events remain associated with their parent tasks, rather than increasing the independent input count.

For the RTX expansion, eligibility and the recipe were frozen before new reference outputs were generated. The exposure audit excluded previously executed or ambiguous inputs and selected 24 MBPP tasks from 224 eligible candidates. No eligible unused HumanEval inputs remained at that stage, so a pre-output amendment selected MBPP. Project-unused means absent from accessible project execution records, not unseen during training. The panel becomes exposed when reused for width studies and later CPU analyses. B570's 15-input panel follows its own accessible-history audit; omitted RTX bulk records were not independently rescanned.

3.2. Registering the output requirement

Let $R(x)$ denote ordinary output under fixed weights, tokenizer, prompt formatting, sampler, runtime, backend and stopping rule. RTX qualification compares the full response array, length, terminal ID and stop reason, and checks reference repeat stability. The expansion contains a complete $24 \times 4 \times 4$ task/condition/repetition matrix: 384 rows and 96 qualified responses per condition. Reference lengths range from 11 to 128 IDs; 23 tasks stop at EOG and one at the cap.

B570 retains IDs through the first runtime EOG or token 128, whichever comes first. A speculative response can satisfy this endpoint requirement while containing additional raw suffix IDs. Those IDs remain charged. Endpoint equality, raw-array equality and unknown historical raw comparisons are reported separately. Historical re-tokenized-text checks retain their original weaker contract (Supplement §S1).

We call a latency ratio a reference-preserving speedup only after its output gate passes. A shorter divergent response does not qualify, and repeat stability alone cannot certify agreement with R. These finite checks establish neither functional correctness nor equality of all intermediate values or arbitrary future outputs.

3.3. Controlled state and current-recipe crosses

At a naturally reached boundary, express the decision as

$$d = F(p, s, e), \tag{1}$$

where p is the committed prefix, s the retained runtime/numerical state and e the current execution recipe. Cross two naturally formed donors s_0, s_1 with recipes e_0, e_1, holding the registered prefix and current logical input fixed. In the resulting matrix, a within-row change is a current-recipe effect for

Table 1: Research context and additional evidence in this study. The cited methods are not same-contract performance baselines.

Prior work	Established basis	Additional evidence here
Batch-invariant inference; LLM-42 [4, 5]	Reduction/shape consistency; verified tokens and KV reconciliation	Qwen3 query views and phase dispatch, finite qualification, and ordinary/matched paid costs
Stage-Replay; Vosti [6, 7]	Fixed-prefix state intervention; specified system determinism	Natural donor × current-recipe crosses with shams; output qualification in a tested implementation family
Consumer cost anatomy; Performance or Illusion? [9, 10]	Backend verification cost and limits of performance proxies	Qualified fixed-UB width failure and separation of target-path from speculative request gains

Table 2: Principal study contracts. Each case processes one request at a time. Historical text checks, which are weaker than direct-ID qualification, are retained separately in the supplement.

Case	Runtime/model setting	Output and exposure	Paid interval
RTX expansion	5070 Ti Laptop; CUDA; Q8_0 target/draft; private b10839-based frontend	Full response through EOG/cap128; 24 project-unused MBPP, four repeats	Server generation; resident HTTP request
B570 repair	Arc B570; Vulkan; Q8_0 target/draft; phase-binding frontend	First-EOG/cap128 endpoint; 15 project-unused inputs, two repeats	Native generation; whole request; host return
M4 runtime	MacBook Air M4; Metal; Q8_0 target/draft	Direct generated IDs; prospective 30 HumanEval plus 30 Spec-Bench, three repeats	Registered post-prefill runtime interval
358H stateful	Arc B390 Vulkan target; symmetric INT4 OpenVINO CPU/NPU draft	Direct IDs; 30 prompts, three repeats per draft condition	Registered stateful interval and target-work accounting

that donor; a within-column change is a donor-state effect for that recipe. Dependence on one factor only under a particular value of the other indicates a local interaction. Token IDs are categories; their numerical differences are not effect sizes.

A valid cross requires consistent cache ownership and positions, faithful restoration, and compatible pending and sampler semantics. Own-path shams restore each donor under its native recipe and compare with uninterrupted execution. Natural donors use online proposals, without feeding future reference answers or a recorded proposal tape into the candidate. Earlier sealed-proposal prediction studies answer a separate reproduction question (Supplement §S2.1). The selected cuts probe known sensitivity; they do not estimate its prevalence or provide a complete counterfactual suffix for every cell.

3.4. Two paid baselines and task-level estimation

Let M be target-only execution on the repaired path and C speculation on that path. Together with ordinary R, they distinguish the deployment change R/C, target-path change R/M and incremental draft benefit M/C. Each baseline is executed and charged. R is the registered comparator, rather than a proven optimum over ordinary implementations.

For late RTX and B570, first take each task's median absolute time in each condition, then form its baseline/candidate ratio. For n complete task pairs,

$$S_{b/c} = \exp\left[\frac{1}{n}\sum_{i=1}^{n}\log\left(t_{i,b}/t_{i,c}\right)\right]. \tag{2}$$

A ratio above one favors the denominator path. For common tasks and this aggregation,

$$S_{R/C} = S_{R/M}S_{M/C}. \tag{3}$$

This multiplicative identity cannot allocate attention, KV, synchronization or host cost. M4 retains its frozen estimator and random streams; 358H uses a median of paired ratios before the task geometric mean, a different estimator.

RTX ordinary-reference ratios are a later CPU reanalysis of the same paid records; the expansion originally emphasized matched comparisons. Descriptive 95% intervals resample the 24 task-median ratios with 10,000 common index resamples, NumPy PCG64 seed 20261008 and linear/type-7 quantiles. The earlier bootstrap convention remains in Supplement §S4.1. Task resampling does not estimate session or day variation, and round-specific geometric means are reported separately from the task-median estimator.

The RTX server-generation field `predicted_ms` charges draft and verification work between its timestamps, excluding initial prompt work and first sampling and potentially final bookkeeping. Resident-request time covers non-streaming HTTP response receipt and parsing. Loading, warmup, boundary telemetry and shutdown are excluded; post-response asynchronous cleanup is not certified inside this interval. Their difference is not a measured host component. B570's native-generation, whole-request and host-return boundaries are defined separately in Supplement §S5.4. Unlike timers are not pooled into a platform ranking.

4. Locating the Consistency Boundary (RQ1)

4.1. A repair and a regression under the same intervention

The continuation experiment tests whether a local path change can recover the reference. The RTX intervention changes target continuation from UB2 to UB1 while retaining prefill, draft

settings and caller grouping. On 19 exposed HumanEval tasks, the predictor reproduces both native conditions in all 57 repeated complete arrays. HumanEval/15 repairs, /25 regresses and /18 remains divergent, leaving reference agreement at 51/57 in both conditions. The unchanged aggregate conceals opposing task effects, so continuation-only control does not satisfy recovery without regression.

A separately selected 24-task MBPP panel reaches 72/72 prediction/native agreement in each condition, but only 66/72 reference agreement. MBPP/304 remains a divergent 47-ID response while changing token 488 to 481 at zero-based position 33. An accurate predictor, a changed response and a restored reference are therefore distinct outcomes. These results motivate examining retained state and the current call separately.

4.2. Natural donor state crossed with the current call

Figure 2 provides the matrix readout. Rows select naturally formed donor states and columns select the current recipe at a shared committed prefix. Horizontal changes expose current-path dependence; vertical changes expose retained-state dependence. The displayed IDs label decisions rather than numerical distances. A change confined to one donor/recipe pairing indicates an interaction. Own-path shams check that restoration itself preserves the native decision.

For RTX, N0 and N1 denote the native baseline and continuation-intervention histories. Four exposed HumanEval probes yield 24 donors, 48 state/current-path forks and 24 shams across repetitions. Donors preserve actual pending state, cache positions and sampler history. The shams pass and forking leaves the original context unchanged. At /15's 13-token cut, either donor gives 9147 under current UB2 and 4199 under UB1 at the third submitted decision (Figure 2A). The local decision follows the current path. At /25's four-token cut, only the N1-donor/current-UB1 cell changes, producing the sequence 271,750,8168 rather than beginning with 1406 (Figure 2B). This is a local state–path interaction. The selected /18 and /31 cuts are discrete nulls, which do not establish invariance over their remaining trajectories.

The KV checks help narrow the design question. All 12 prompt-KV donor comparisons agree, whereas 70 of 72 generated-prefix K/V tensor comparisons differ; the tested first-layer K/V remain equal. The discriminating /15 bonus row is itself a single-token microbatch under either history. Its final nominal shape alone therefore cannot explain the decision: inputs and preceding state formation also matter. The evidence motivates attention-related controls without isolating a unique kernel cause. Auxiliary short continuations and their 1,278 forwards are diagnostic work, not a paid performance panel.

4.3. One unchanged block does not certify the remaining trajectory

Immediate agreement need not certify the remaining trajectory. A separate B570 experiment forks the same live state, with byte-checked KV, at the first eligible verification block after at least 64 committed IDs. It processes that block under UB1 or UB128. Twenty-four reserved prompts and three repetitions produce 72 eligible forks. Every paired raw score record changes, yet immediate top-1 IDs and acceptance decisions remain equal across 195 paired decision events: no execution or task changes its immediate decision.

Continuation gives a different answer (Figure 2F). Complete committed-ID trajectories differ in 18/72 executions from six of 24 tasks. Later alternate blocks continue to use UB128, and proposals evolve under the accepted history. This result concerns a sustained continuation policy, not one altered block followed by restoration. The practical lesson is to qualify the registered output beyond the sampled block: a local discrete null can coexist with later divergence, and score movement alone does not predict an immediate decision change. Neither the 72 forks nor the 195 internal events constitute additional independent tasks.

4.4. B570 crosses identify a whole-recipe boundary

For B570, N1 labels the repaired prior frontend. The figure uses E for the alternate donor: the cross recipe E_k1_ub2 on HumanEval, but ordinary N0 on MBPP/72. Neither denotes the later phase-binding candidate. These crosses test whether the RTX diagnostic distinction remains useful in a different runtime. Three tasks in six repeated contexts provide 12 own-path shams and 24 auxiliary target cells. Restored states retain checked byte contents, sequence-zero ownership and logical positions. At HumanEval/145's prefix 56, the selected matrix is $[(71465, 279), (279, 279)]$; at /42's prefix 73 it is $[(432, 1817), (1817, 1817)]$ (Figure 2D,E). Both show a local interaction. At MBPP/72's prefix 40, $[(320, 320), (308, 308)]$ varies only with donor state, under identical current UB1/output-reservation-3 columns (Figure 2C).

The HumanEval recipes change microbatch capacity and output reservation together, so the factor tested is the whole recipe. These crosses do not isolate UB or transfer the RTX repair to Vulkan. Full counterfactual continuations for each cell are unavailable; the subsequent B570 candidate undergoes its own native endpoint qualification. The greedy sampler history, clone metadata and shams support the tested restoration, while opaque sampler bytes and general stochastic-state restoration remain outside that check. Across the cases, the same committed prefix can conceal path, state or interaction dependence, requiring different control placements.

4.5. Prediction and simple execution constrain added machinery

Prediction fidelity alone does not establish an online repair mechanism. B570 prospective prediction matches 72/72 native arrays on a reserved 24-task cohort even though reference agreement is 60/72. RTX's initial target-only batched-path predictor reaches 24/36 and fails its frozen 36/36 gate, so no reserved inference follows. A later independent-proposal replay obtains H2/native and reference-only/native agreement of 72/72, again with 60/72 reference agreement. The later study resolves the earlier reproduction interface, but neither result establishes cheap online correction.

A simpler target-only bridge strengthens that distinction. On the 19 RTX tasks, draft-free U_2 reproduces all 57 intervention

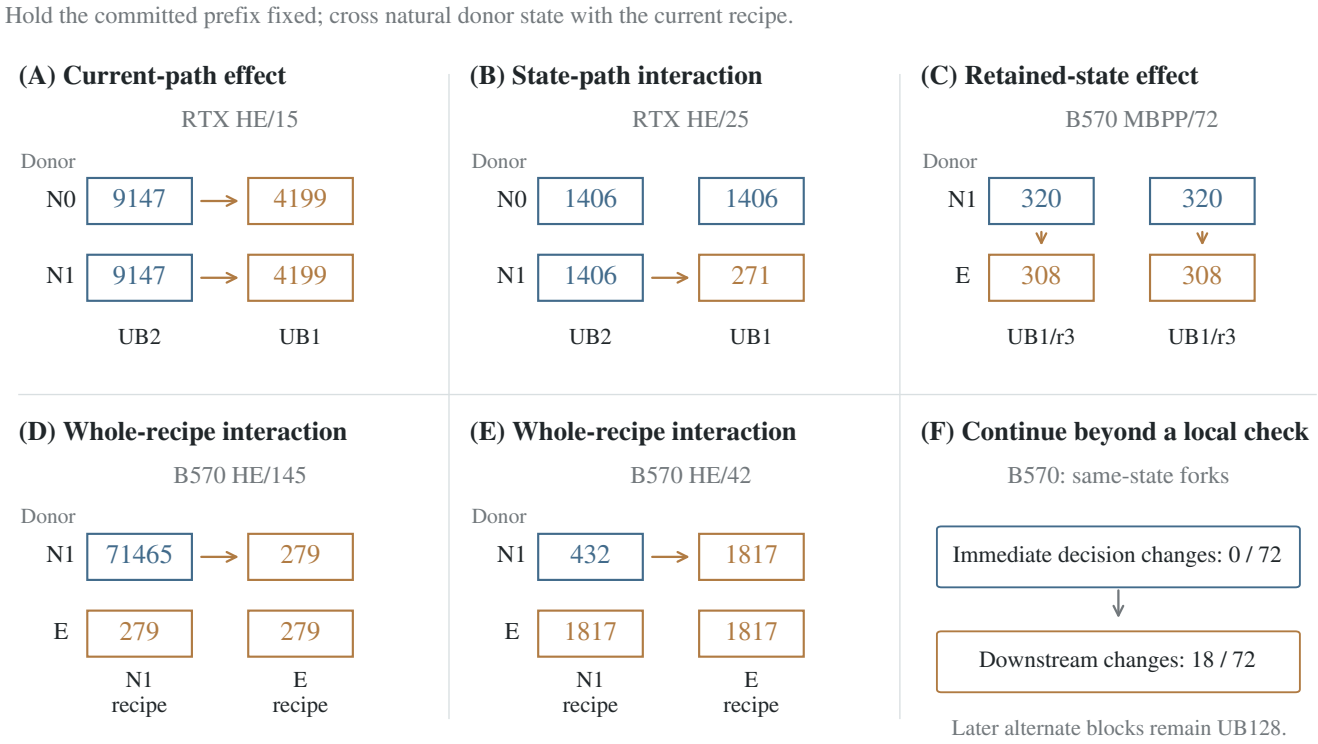


Figure 2: Three local dependencies and the limit of a local check. Rows select natural donor states; columns select the current recipe. Token IDs are categorical, with colors grouping equality within each case. (A) RTX /15 follows the current path. (B) RTX /25 and (D,E) B570 HumanEval show interactions. (C) B570 MBPP/72 follows retained state under identical UB1/reservation3 columns. The B570 HumanEval columns change UB and output reservation together. E denotes the alternate donor: ordinary N0 in (C), the alternate cross recipe in (D,E). Own-path shams pass. (F) A separate B570 study finds 0/72 immediate but 18/72 downstream changes under sustained alternate UB128 continuation; it does not isolate a single-block aftereffect.

N_1 arrays and stop boundaries, including /15's repair and /25's regression. It needs no draft, transplant, proposal tape or reference answer. This establishes output equivalence, without a speed claim or demonstrated equality of live logits/KV.

The offline selection study also supplies no supported advantage for added screening. RTX direct enumeration and proxy screening both abstain after 114 record equivalents beyond common acquisition; B570 direct verification requires 72 and proxy screening 144. Unknown counterfactuals remain unknown. These are evidence-acquisition counts, not latency measurements. RQ1 is thus answered by local diagnostic distinctions; it does not yield a validated automatic repair selector.

5. Phase-Scoped Implementation and Qualification (RQ2)

5.1. Constraining attention while retaining batched work

The RTX design targets the attention graph rather than serializing the whole network. An initial implementation builds one non-flash Qwen3 attention subgraph per query and concatenates the results. The later query-batch implementation represents query columns in the ggml batch dimension, allowing existing operations to process them in parallel while preserving one-column query structure. Projection, output projection and feed-forward computation remain batched (Figure 1B).

After the existing query/key/value permutations, the supported single-KV-stream query has dimensions $(d, q, h, 1)$. A `ggml_view_4d` reinterprets it as $(d, 1, h, q)$, preserving the source strides and assigning the original query-column stride to the new batch axis. Shared K/V broadcast across query batches; the mask view changes from $(n_{\mathrm{KV}}, q, 1, 1)$ to $(n_{\mathrm{KV}}, 1, 1, q)$. Existing key-query multiplication, softmax, value multiplication and contiguous recombination perform attention. The design retains the tested scalar-query computation structure locally while preserving parallel work elsewhere; it adds no new backend arithmetic.

Implementation guards admit Qwen3 non-flash attention, one KV stream and compatible mask dimensions, with no attention bias, sinks or MLA value projection. The initial limit is four queries; a later width build extends the admitted graph to eight. Native qualification is required within those guards, and unsupported model graphs, precisions or backends inherit no preservation claim.

5.2. Phase dispatch and graph reuse are part of the implementation

The transformation must be active in the intended phase, including when diagnostic capture is disabled. A process-fixed mask and thread-local guard select target prefill/continuation bits 1/2 and draft prefill/continuation bits 4/8 (Figure 1C). The guard

restores the prior active state on exit and executes before the capture-disabled fast path returns to native decoding. Diagnostic settings therefore cannot silently bypass the constraint.

The active query-batch flag is copied into graph parameters and context. Reuse is rejected if it differs from the cached topology key, preventing a phase-layout mismatch even when ordinary microbatch fields agree. A new prompt resets the phase to prefill. These conditions make role and phase part of graph construction and reuse, rather than annotations on a timing record.

Qualification checks captured multi-query nodes and capture-on/off behavior; paid capture-off sessions omit observer metadata and forced witness synchronization. The optional target-state-move branch is inactive in the expansion configuration. The measured candidate charges native drafting, verification and reconciliation without that transplant. Implementation details and timer boundaries are retained in Supplement §S3.1.

5.3. The finite phase family

The factorial tests which phases require the query-batch constraint in this implementation. Figure 3A tests all 16 masks at $k = 3$/UB4 on four exposed HumanEval probes, twice each: 128 requests. Outputs and stop boundaries are repeat-stable. Masks 3, 7, 11 and 15 pass the complete panel. /15 and /31 pass every mask, /25 needs at least one target-phase bit, and /18 needs both; draft bits do not change this pattern.

Mask 3 alone has no passing strict subset, making it inclusion-minimal in the tested four-switch family. This selects a sufficient target-phase scope, not a universal requirement on every input or implementation. Acceptance, cache ownership and stopping still need to preserve the transaction. The factorial also uses different geometry from the earlier UB2/UB1 intervention, so /15's all-mask agreement does not negate that path effect.

A smaller scope need not be faster. In a separate 19-task, six-repetition cost panel, mask 3 gives 1.278× generation and 1.113× request speed against matched target-only. Removing draft constraints from mask 15 adds only 1.004× generation speed, with descriptive interval [0.998, 1.009], below the frozen 1.03× extra-generation gate. The later 24-input expansion gives 1.015× over mask 15. Scope selection therefore follows qualification, with its cost assessed separately.

5.4. Frozen inputs and geometry boundaries

The frozen mask-3/$k = 3$/UB4 recipe qualifies all 96 candidate responses on the 24-input expansion. Matched target-only and mask-15 speculation also qualify. This extends the repair from exposed probes to a separately frozen panel, within the same model family and configuration.

Figure 3B shows where that sufficiency breaks. An independent 11-input panel tests $(k, \mathrm{UB}) = (1, 2), (3, 4), (5, 6), (7, 8)$ at caps128/256/512, with fresh scalar references and two repeats. The first two pairs qualify; MBPP/229 fails at both wider pairs across all caps and repetitions while matched controls pass. The earlier HumanEval/18 wide target-only failure remains a separate boundary outside this denominator. The longest observed reference has 261 IDs, so cap512 is a limit rather than evidence of a 512-token trajectory.

At MBPP/229's common 47-token prefix, crossing naturally retained ordinary and constrained states under two current UB6 layouts changes the EOG decision with the donor. Current layout changes continuous scores without a separate discrete flip; four own-path shams pass. This local retained-state effect explains why changing only the current layout at that boundary is insufficient, without identifying a unique cause of all width failures. Two cap contexts repeat one task boundary. The joint k/UB sweep remains distinct from the fixed-UB draft-width cost experiment in Section 6.

5.5. A separate B570 implementation and qualification scope

B570 tests a different control interface. The candidate forms target state with the ordinary prompt caller and output reservation, samples the pending token once, then binds continuation from UB128 to UB1 in the same context and live sampler. A private API synchronizes, changes capacity and requests scheduler reservation under a frozen context/batch/sequence contract. Instrumented checks compare serialized KV across binding; paid capture-off requests omit witness readbacks. The existing $k = 2$, draft-UB1 accept/bonus/rollback loop remains.

The candidate qualifies 12 development requests, 112 requests on 56 exposed inputs and 30 requests on 15 project-unused inputs under its endpoint contract. It removes the old second context and three witness readbacks but does not implement the RTX query-layout transformation. Earlier B570 repair produced a local long-panel gain yet failed MBPP/72 on a new-input panel and five UB2 alternatives (Supplement §S5.1). The later same-context binding supplies a separately qualified repair; its engineering value depends on the paid comparisons below.

6. Does the Qualified Design Pay? (RQ3)

6.1. RTX: deployment gain and speculative increment

The RTX comparison separates two decisions before measuring their benefit (Figure 4A). Ordinary target-only R uses mask 0/UB1; matched target-only M uses mask 3/UB4; speculative C, also denoted C_3, adds a draft at $k = 3$ on that path. A fourth condition, C_{15}, constrains all phases. Each of four balanced rounds uses a separate resident process for each condition, with one excluded warmup and 24 formal requests per process. Condition and task orders rotate and recorded cache reuse is zero. The comparisons share tasks and a campaign, without sharing one simultaneous resident process.

Table 3 and Figure 4B,C show why both baselines matter. Generation speed is 1.170× ordinary and 1.169× matched target-only, with 22/24 tasks faster in either comparison. Resident-request speed is 1.742× ordinary but 1.035× matched. The target-only path change itself gives $R/M = 1.683\times$ request speed. Thus most of the measured deployment improvement is already available without a draft; 1.742× describes the complete change, whereas 1.035× is the speculative increment on the matched path.

The identity in Equation 3 holds for all 48 task-timer cells and both aggregate timers. Generation R/M is near parity while request R/M is much larger. The target-path change affects

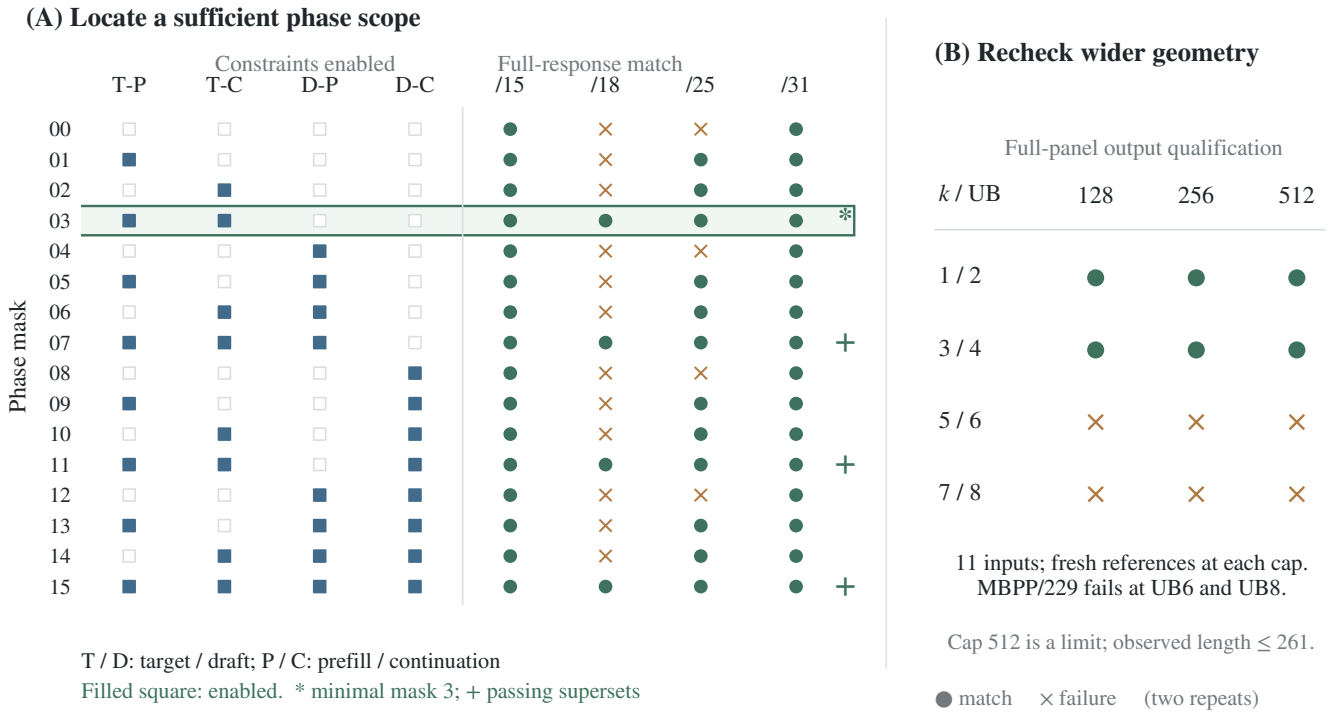


Figure 3: Qualification of phase scope and geometry. (A) All 16 masks at $k = 3$, UB4 and cap128 on four HumanEval probes, with two repetitions (128 requests). Filled squares enable constraints; circles indicate full-response matches and crosses failures. Passing masks are 3/7/11/15; mask 3 is inclusion-minimal within this four-switch family. (B) A separate 11-input panel changes k and UB jointly at caps128/256/512, using fresh scalar references and two repeats. MBPP/229 fails at UB6/8 while matched controls pass. Actual output length reaches at most 261; the earlier HE/18 target-only failure is outside this panel.

Table 3: RTX frozen expansion paid ratios: 24 task medians, four repeats. Gen. denotes generation; Request denotes resident request. Intervals describe task resampling. C_3 is mask 3 speculation.

Ratio	Timer	Speed	95% interval
R/C_3	Gen.	1.170	[1.117,1.218]
R/C_3	Request	1.742	[1.655,1.827]
M/C_3	Gen.	1.169	[1.117,1.217]
M/C_3	Request	1.035	[1.005,1.063]
R/M	Gen.	1.001	[0.995,1.008]
R/M	Request	1.683	[1.590,1.766]

prefill microbatch/attention behavior and continuation capacity jointly, and the timer boundaries differ. Their contrast cannot allocate a measured host, attention, KV or synchronization share. A faster qualified ordinary implementation could also narrow the deployment benefit: the registered R is an executed reference, not a proven fastest ordinary decoder.

6.2. *Task variation and session uncertainty*

The matched-path benefit varies across tasks and rounds. Figure 5A,B retains all 24 tasks in the same request-sorted order and scale. Generation improves on 22/24 tasks, but request time improves on only 17/24. Seven tasks regress against matched execution, despite all 24 being faster than ordinary request cost. The average request gain therefore does not justify an unconditional per-task acceleration claim.

Figure 5C exposes a second limitation. The within-round matched-request geometric means are 1.029, 0.940, 1.052 and 1.044: round 2 reverses direction. These round summaries differ from the geometric mean formed after taking each task's four-repetition absolute-time medians. The descriptive task-bootstrap interval for that median-based aggregate lies above one, but does not remove the observed round reversal or estimate independent session/day uncertainty.

The operating point is qualified and positive in aggregate, with a small, uneven request increment. Separate resident sessions, laptop scheduling and thermal variation can affect that estimate; balanced order and task medians do not eliminate session confounding. The larger generation gain applies to its defined interval rather than replacing the request result. Independent session/day replication would be needed to assess stable deployment acceleration.

A separate serial resident-reset study checks whether the anchor preserves output when reused across requests. Two orders retain the registered reference after target-state reset, with matched generation/request speeds about 1.181 × /1.039×. This supports finite reset semantics for the original $k = 3$/UB4 recipe, without establishing concurrency, throughput or resident qualification of the widened draft. These rows are not pooled with the expansion.

6.3. *Fixed UB: wider drafting qualifies but becomes slower*

The fixed-UB experiment tests the cost of widening a qualified draft. Unlike the joint k/UB geometry sweep, this experiment

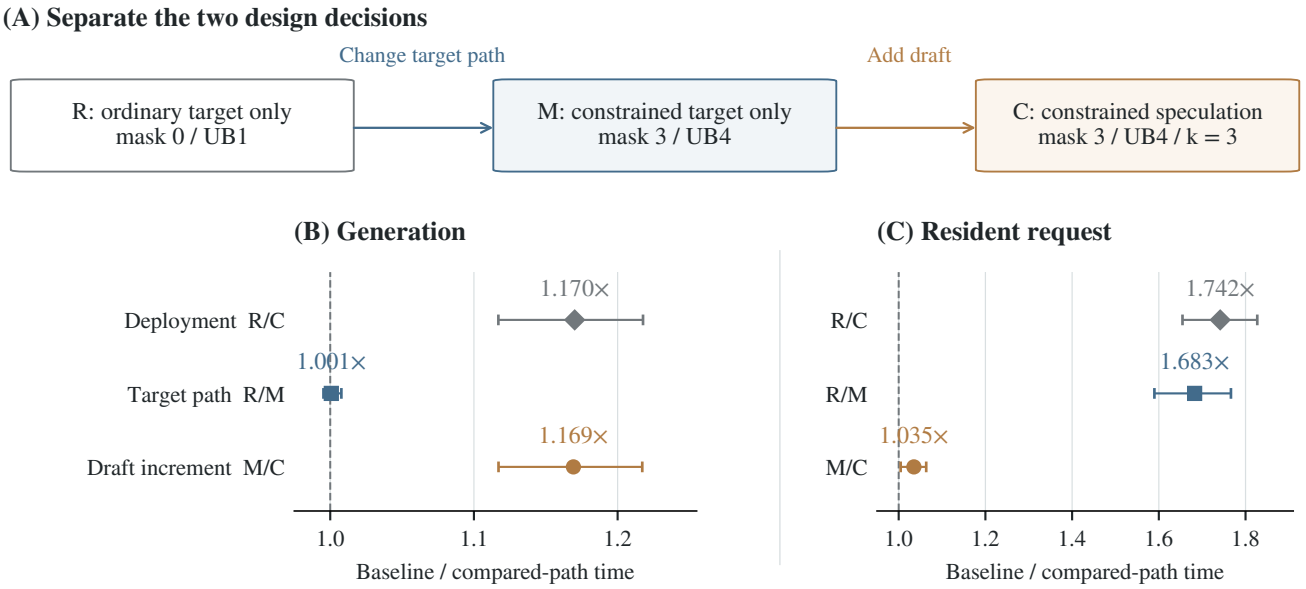


Figure 4: Deployment gain and incremental draft benefit. (A) R is ordinary target-only execution, M the repaired target-only path, and C speculation on that path. (B,C) Geometric means of 24 task ratios formed from four-repetition absolute-time medians; bars show descriptive 95% task-bootstrap intervals. A ratio above one favors the denominator. Generation and resident-request timers use different intervals and axis ranges. Request ratios satisfy $R/C = (R/M)(M/C)$; this multiplication separates comparisons rather than component costs.

fixes UB4 and mask 3 while testing $k = 3, 5, 7$ on 24 now-exposed MBPP inputs and four exposed HumanEval probes. Each width qualifies in two repetitions. The highest passing width, $k = 7$, is then measured in a separate capture-off arm: four conditions, four repetitions and 448 formal requests. Qualification timings do not select the width by speed.

The qualified wider draft loses the cost comparison. Relative to $k = 3$, $k = 7$ gives 0.829× generation and 0.890× request speed. Against matched target-only it gives 0.997 × /0.950×, whereas the anchor gives 1.203 × /1.068× in this campaign. Ordinary same-width speculation fails its output gate, so its timing cannot serve as qualified speed evidence. For these inputs and this configuration, the measured anchor is preferable to the wider qualified draft.

The counters explain why caller reduction is insufficient as a selection rule. Drafted tokens rise from 3,528 to 5,392, while accepted draft tokens rise from 2,772 to 3,192. Separate captured qualification sessions record target continuation caller counts falling from 305 to 199 but micrographs increasing from 305 to 397. These counts include their own warmups and forced synchronization; they cannot be divided into formal timing to create a component profile. Micrographs are not CUDA kernel launches, and no unique slowdown component is isolated.

The saved partitions further limit the pooled result. At $k = 7$, matched generation/request speed is 0.953 × /0.919× on 24 MBPP inputs and 1.308 × /1.164× on four HumanEval probes. Four exposed probes do not validate an online routing rule. Supplement §S4.3 retains these partitions and the longer-cap checks under their own timers and panels.

6.4. *B570: repair improves an old path but loses to strong baselines*

The B570 repair must pass both the output and cost tests. Its formal four-condition matrix contains 119/120 payloads; old N1's first-round MBPP/400 record is missing after an infrastructure interruption. Its time remains unknown, without imputation or a scientific rerun. Ordinary R, matched M and candidate C each have 30 complete responses and 15 two-round task-median pairs. Old-N1 comparisons use 14 complete pairs.

The candidate fails the latency objective against both strong baselines (Table 4). Generation/whole-request speed is 0.283 × /0.201× against ordinary R and 0.286×/0.206× against matched target-only. Both baselines are faster in every available paired round. The target-only paths are close in cost, with R/M = 0.989 × /0.979×; the roughly fivefold candidate request cost is therefore not explained by the target-path consistency constraint alone.

Against the prior old-N1 frontend, the candidate improves generation/request speed by 3.993 × /2.634× over 14 pairs. That improvement measures progress from a slow frontend, rather than benefit over ordinary deployment. MBPP/245 also reverses the old-path whole/host request comparison in round 2. Endpoint recovery and aggregate improvement do not guarantee improvement in every round.

The endpoint contract retains all online work. Candidate endpoint IDs total 1,150 while raw IDs total 1,176, with all 26

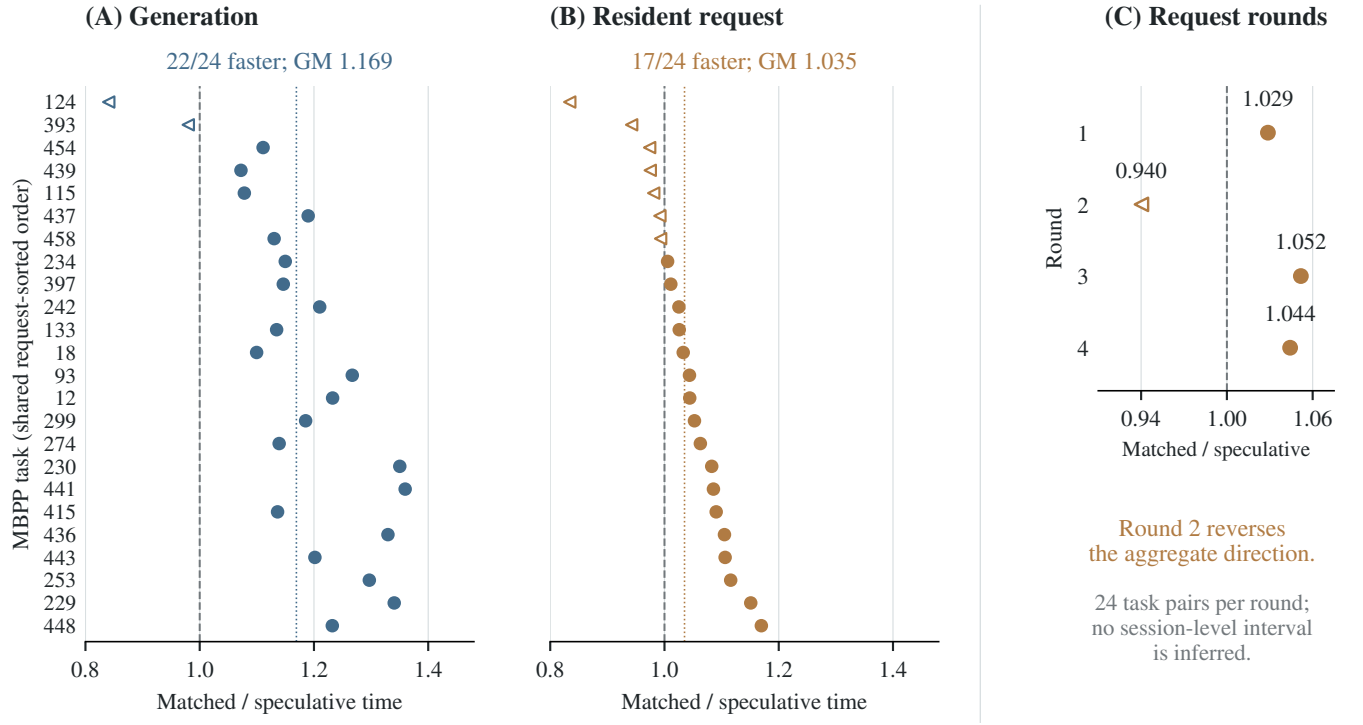


Figure 5: Variation behind the matched RTX benefit. (A,B) All 24 tasks share order and scale; ratios use four-repetition absolute-time medians. Generation improves on 22/24 and request time on 17/24. Filled circles denote faster speculation and open left triangles slower speculation, separately for each timer. Dashed lines mark parity; dotted lines mark task-median geometric means. (C) Geometric means within four request rounds, with reversal in round 2. Round summaries are not the task-median estimator or a session-level uncertainty interval.

Table 4: B570 paid baseline/candidate task-median ratios. n counts complete two-round task pairs: 15 for R/C, M/C and R/M, and 14 for old N1/C. Gen., Request and Host denote generation, whole request and host-observed time. The missing old-N1 task is not imputed; raw suffix work stays charged.

Ratio	n	Gen.	Request	Host
R/C	15	0.283	0.201	0.202
M/C	15	0.286	0.206	0.206
Old N1/C	14	3.993	2.634	2.633
R/M	15	0.989	0.979	0.979

suffix IDs charged. The records contain 888 drafted and 702 accepted tokens, and target/draft continuation caller-token totals of 1,332/1,362. These quantities describe different parts of the transaction; endpoint equality does not assert raw suffix equality. Removing witness readbacks, an extra context and handoff improves the frontend, but phase reservation, synchronization, drafting and rollback remain. Without a full component profile, the remaining bottleneck cannot be assigned uniquely. The paid decision is to reject this candidate for latency against R and M.

6.5. *Global serialization, control, and draft-side limits*

The other campaigns test three tempting shortcuts. First, global serialization can recover output while increasing cost. An earlier RTX arm qualifies 57/57 arrays on 19 exposed tasks but achieves 0.677× target-only speed, with descriptive interval $[0.662, 0.692]$ and no task faster. Its synchronized post-prefill-to-slot-release timer includes instrumentation and logging. It is a separate negative operating point, not an earlier point on the expansion's server-generation curve; Supplement §S6.4 retains the host amendment and preceding V3 stop.

Second, added control must pay for itself. The prospective M4 campaign has 60 tasks, three conditions, three repeats and 540 requests under a post-prefill contract. Always speculation and Guard each qualify 180/180 direct-ID comparisons. Target-only/Always speed is 1.347×, but the paired secondary Guard/Always latency ratio is 1.027, with descriptive interval $[1.013, 1.044]$: the measured guard is slower. A hindsight oracle choosing only target-only or Always offers about 0.069% further geometric-mean latency reduction. This narrow two-action opportunity does not bound intra-request policies; the primary and secondary statistical streams remain distinct (Supplement §S6.6).

Third, cheaper drafting may leave a target-work floor. On

358H, CPU/NPU drafting each qualifies 90 direct-ID comparisons but gives 0.734 × /0.352× speed. With measured target work and committed progress fixed, eliminating every other non-negative cost yields prompt-estimator ceilings of 0.837×/0.845×. Even that idealization needs a uniform target-work reduction of at least 16.28%/15.52% to reach parity. Draft-only optimization cannot close this conditional budget; changing the target path or state interface requires new qualification and measurement. These are necessary budgets, not achieved optimizations or an impossibility result for NPUs (Supplement §S6.7). M4 and 358H inform separate design decisions and do not replicate the RTX repair.

7. Design Implications and Limits

7.1. Constrain the transaction where evidence warrants it

The diagnostic distinction guides control placement. A current-path effect motivates constraining verification geometry; a retained-state effect motivates preserving state formation or changing its interface; an interaction requires testing both. The RTX design applies target-phase attention constraints, whereas B570 binds a whole recipe within one context. These are distinct implementations informed by local observations, without a unique kernel diagnosis or an automatic selector.

The sequence of checks is equally important. Own-path shams validate restoration at the tested boundary; crosses reveal local dependence; complete registered-output checks qualify the candidate. B570's unchanged immediate decisions and changed continuation make the last step essential. Cost follows qualification: constraining fewer phases or accepting a wider draft can still fail the performance objective.

7.2. Use both paid baselines for operating decisions

The ordinary comparison answers whether the deployment change helps; the matched comparison answers whether adding a draft helps on the repaired path. RTX exposes a substantial draft-free target-path opportunity followed by a modest, uneven request increment. Both values are needed to choose an implementation. A faster qualified ordinary decoder could reduce the former, since the registered R is not a demonstrated optimum.

The negative results also determine operating choices. Retain the measured RTX anchor rather than its slower qualified wider draft; reject the B570 recipe for latency against both strong baselines; avoid adding the measured M4 guard on an assumed control benefit; and address 358H target work or state interfaces before expecting draft-only optimization to reach parity. Each decision follows its own output and timer contract, rather than a pooled hardware ranking.

7.3. Scope of the evidence

The repair covers one Qwen3 target/draft family, finite panels and pinned runtime variants. RTX's project-unused panel consists of 24 selected MBPP inputs; unused refers to accessible project history, not training exposure. Changed geometry needs renewed qualification, as the failures demonstrate. No second-family result or preservation guarantee on arbitrary inputs is established.

Discrete agreement is weaker than numerical equivalence. The score/KV witnesses do not separate every reduction, precision, layout or scheduling effect, and shams cannot exhaust hidden backend state. Selected cuts and a four-switch family cannot establish universal minimality. Prior preservation methods were not implemented as same-contract baselines, so superiority over them remains untested.

Task medians and descriptive intervals leave session, thermal and infrastructure variation unresolved. The B570 old-N1 payload and some historical raw comparisons remain missing. Timer and historical energy domains differ; later direct-ID qualification cannot validate earlier text-based timing. The reproducible CPU checks cover included sources and exports, with original-host private-archive audits identified separately in Supplement §S7.

8. Conclusion

A runtime that must reproduce an ordinary greedy output needs to preserve the stateful transaction as well as its acceptance rule. In the tested RTX implementation, a query-layout change during target prefill and continuation retains other batched work and qualifies a finite full-response panel. Natural crosses motivate that scope, while the complete phase matrix and wider-geometry failures bound it.

Paid ordinary and matched comparisons then change the performance interpretation: 1.742× RTX request speed includes a 1.683× draft-free target-path gain, leaving 1.035× from speculation on that path. The increment is uneven and reverses in one round. A wider qualified draft loses, B570 endpoint repair remains slower than strong baselines, and the separate M4/358H cases constrain the value of added control and draft-only optimization. These outcomes support a concrete design practice within the studied family: diagnose local dependence, qualify the complete output under an explicit configuration, then charge both baselines before selecting speculation.

Author-review declarations

Authorship and contributions. Author names, affiliations, corresponding-author details and individual CRediT roles await author confirmation. This draft does not assign authorship or contributions.

Funding and competing interests. Funding sources and competing-interest declarations await confirmation by all authors; absence of a funding source or competing interest is not asserted.

Data and software availability. The author-review source package includes frozen committed evidence exports, source bindings, selected implementation and report records, and CPU-only code for recalculation and figures. Original runtime code and experiment records are retained in the HeteroSpec project. Availability and permitted redistribution of private raw archives,

model artifacts and a permanent public data/software location require author confirmation before submission. The supplied exports enable the numerical checks specified in Supplement §S7; they do not substitute for a full private-archive audit or a rerun of model inference.

Declaration of generative AI and AI-assisted technologies in manuscript preparation. OpenAI ChatGPT/Codex was used to assist with restructuring and drafting text, source and bibliography checks, CPU analysis and plotting code, and the execution-architecture illustration. Quantitative figures are generated from the identified experimental exports, without generative replacement of observations. This author-review version still requires the authors' final scientific, bibliographic and language verification. The final declaration and responsibility statement must be confirmed by the authors before submission.

References

[1] Y. Leviathan, M. Kalman, Y. Matias, Fast inference from transformers via speculative decoding, in: Proceedings of the 40th International Conference on Machine Learning, Vol. 202, PMLR, 2023, pp. 19274–19286, accessed 9 October 2026.
URL https://proceedings.mlr.press/v202/leviathan23a.html

[2] ggml-org llama.cpp contributors, llama.cpp [software], Releases b10809 and b10839; private study patches included in the author source package, b10809 released 4 September 2026; b10839 released 7 September 2026. Accessed 9 October 2026 (2026).
URL https://github.com/ggml-org/llama.cpp/releases/tag/b10839

[3] A. Yang, et al., Qwen3 technical report, arXiv preprint arXiv:2505.09388v1, accessed 9 October 2026 (2025). doi:10.48550/arXiv.2505.09388.
URL https://arxiv.org/abs/2505.09388v1

[4] H. He, Thinking Machines Lab, Defeating nondeterminism in llm inference, Thinking Machines Lab: Connectionism, accessed 9 October 2026 (2025). doi:10.64434/tml.20250910.
URL https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/

[5] R. Gond, A. K. Kamath, R. Ramjee, A. Panwar, LLM-42: Enabling determinism in LLM inference with verified speculation, arXiv preprint arXiv:2601.17768v2, accessed 9 October 2026 (2026). doi:10.48550/arXiv.2601.17768.
URL https://arxiv.org/abs/2601.17768v2

[6] A. B. Lorup, Stage-replay divergence follows the KV cache: Fixed-prefix precision controls and bidirectional cache transplantation, arXiv preprint arXiv:2607.28495v1, accessed 9 October 2026 (2026). doi:10.48550/arXiv.2607.28495.
URL https://arxiv.org/abs/2607.28495v1

[7] J. Qin, A. Du, D. Zhang, M. Lentz, D. Zhuo, Vosti: Specifying, implementing, and verifying deterministic llm inference, arXiv preprint arXiv:2609.38981v1, accessed 9 October 2026; arXiv reports DOI registration pending (2026). doi:10.48550/arXiv.2609.38981.
URL https://arxiv.org/abs/2609.38981v1

[8] K. Chu, Y. Zhou, W. Zhang, MarginGate: Sparse margin-triggered verification for batch-invariant LLM inference, arXiv preprint arXiv:2605.30218v1, accessed 9 October 2026 (2026). doi:10.48550/arXiv.2605.30218.
URL https://arxiv.org/abs/2605.30218v1

[9] P. Chordiya, Lossless but not free: An empirical anatomy of speculative decoding on consumer hardware, arXiv preprint arXiv:2607.17283v1, accessed 9 October 2026 (2026). doi:10.48550/arXiv.2607.17283.
URL https://arxiv.org/abs/2607.17283v1

[10] X. Liu, J. Yu, J. Park, I. Stoica, A. Cheung, Speculative decoding: Performance or illusion?, in: Proceedings of Machine Learning and Systems, Vol. 8, 2026, accessed 9 October 2026.
URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/554e056fe2b6d9fd27ffcd3367ae1267-Abstract-Conference.html

[11] L. Zhang, Z. Zhang, B. Xu, R. Li, Z. Tian, S. Mei, D. Li, Dovetail: A CPU/GPU heterogeneous speculative decoding for LLM inference, arXiv preprint arXiv:2412.18934v2, accessed 9 October 2026 (2025). doi:10.48550/arXiv.2412.18934.
URL https://arxiv.org/abs/2412.18934v2

[12] K. Lv, H. Guo, Q. Guo, X. Qiu, DuoDecoding: Hardware-aware heterogeneous speculative decoding with dynamic multi-sequence drafting, arXiv preprint arXiv:2503.00784v1, accessed 9 October 2026 (2025). doi:10.48550/arXiv.2503.00784.
URL https://arxiv.org/abs/2503.00784v1

[13] X. Liu, J. Park, L. Hu, W. Kwon, Z. Li, C. Zhang, K. Du, X. Mo, K. You, A. Cheung, Z. Deng, I. Stoica, H. Zhang, TurboSpec: Closed-loop speculation control system for optimizing LLM serving goodput, arXiv preprint arXiv:2406.14066v3, accessed 9 October 2026 (2025). doi:10.48550/arXiv.2406.14066.
URL https://arxiv.org/abs/2406.14066v3

[14] Y. Hou, F. Zhang, C. Du, X. Zhang, J. Pan, T. Pang, C. Du, V. Tan, Z. Yang, BanditSpec: Adaptive speculative decoding via bandit algorithms, in: Proceedings of the 42nd International Conference on Machine Learning, Vol. 267, PMLR, 2025, pp. 24045–24079, accessed 9 October 2026.
URL https://proceedings.mlr.press/v267/hou25h.html

[15] S. Liu, Y. Ye, Q. Zhu, Z. Cao, Y. He, HeteroSpec: Leveraging contextual heterogeneity for efficient speculative decoding, in: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, 2026, pp. 12930–12947, accessed 9 October 2026. doi:10.18653/v1/2026.acl-long.589.
URL https://aclanthology.org/2026.acl-long.589/

[16] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. Petroski Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. Hebgen Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, W. Zaremba, Evaluating large language models trained on code, arXiv preprint arXiv:2107.03374v2, accessed 9 October 2026 (2021). doi:10.48550/arXiv.2107.03374.
URL https://arxiv.org/abs/2107.03374v2

[17] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, C. Sutton, Program synthesis with large language models, arXiv preprint arXiv:2108.07732v1, accessed 9 October 2026 (2021). doi:10.48550/arXiv.2108.07732.
URL https://arxiv.org/abs/2108.07732v1

[18] H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, Z. Sui, Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding, in: Findings of the Association for Computational Linguistics: ACL 2024, Association for Computational Linguistics, 2024, pp. 7655–7671, accessed 9 October 2026. doi:10.18653/v1/2024.findings-acl.456.
URL https://aclanthology.org/2024.findings-acl.456/